

A JAVA-FIRST FIELD GUIDE

Mastering System Design — LLD Java

Object-oriented design, SOLID and friends, concurrency & thread-locking, UML notation, the full Gang-of-Four pattern catalog, and sixteen complete FAANG-style low-level-design case studies — worked file-by-file, in Java.

Language: Java 17

Level: Beginner → Interview-ready

Edition: Complete, single volume (with Concurrency & 16 LLD Case Studies)

Table of Contents

CHAPTERS / page numbers match the PDF viewer

Chapter 1	What is LLD? LLD vs HLD	5
Chapter 2	The Four Pillars of OOP, in Java	6
Chapter 3	Design Principles: SOLID and Practical Simplicity	24
Chapter 4	UML: Diagram Types & Notation	39
Chapter 5	Creational Design Patterns	41
Chapter 6	Structural Design Patterns	52
Chapter 7	Behavioral Design Patterns	67
Chapter 8	Concurrency & Thread Locking in Java	91
Chapter 9	How to Approach an LLD Problem in 60 Minutes	95
Chapter 10	Movie Ticket Booking System	97
Chapter 11	Inventory Management System	102
Chapter 12	Parking Lot System	105
Chapter 13	Notification System	109
Chapter 14	Rate Limiter	112
Chapter 15	Logger Framework	115
Chapter 16	Train Reservation System	118
Chapter 17	Car Rental System	121
Chapter 18	Expense Sharing App	124
Chapter 19	Live Cricket Score System	127
Chapter 20	Coupon / Discount Engine	131
Chapter 21	Ride Booking System	134
Chapter 22	Payment Gateway	137
Chapter 23	Video Conferencing System	140
Chapter 24	E-commerce Order Management System	143
Chapter 25	Food Delivery System	146
Chapter 26	Final Interview Tips	150
Notes	Sources and verification	152

PART 1 — FOUNDATIONS

Chapter 1: What is LLD? LLD vs HLD

Every real system gets designed twice. First at the level of services, data stores, and how machines talk to each other — that's **High-Level Design (HLD)**. Then at the level of classes, interfaces, and how objects inside one service collaborate — that's **Low-Level Design (LLD)**. HLD tells you a ride-sharing app needs a matching service, a pricing service, and a Postgres cluster behind a load balancer. LLD tells you, inside the matching service, what a `Driver` class looks like, how a `MatchingStrategy` is chosen at runtime, and which interface a new matching algorithm has to implement to slot in without touching existing code.

LLD is the bridge between an architecture diagram and code that actually compiles. It answers questions HLD deliberately leaves open: which class owns which responsibility, how state is protected from misuse, and how the design stays open to new requirements without a rewrite.

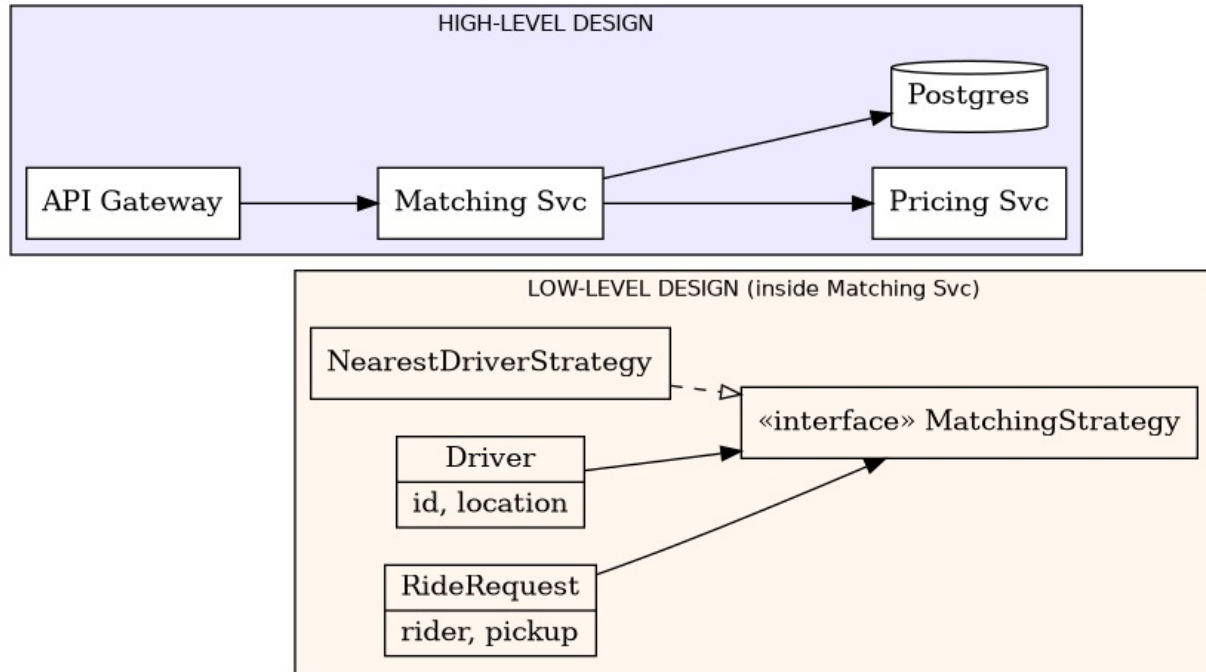


Fig 1.1 — HLD picks the services and stores; LLD picks the classes and how they collaborate inside one of those services.

Side-by-side comparison

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Granularity	Services, modules, data stores	Classes, interfaces, methods, fields
Typical output	Component & deployment diagrams	Class & sequence diagrams, code
Key concerns	Scalability, availability, network	Cohesion, coupling, extensibility
Who owns it	Architects, senior engineers	Every engineer implementing a feature
Tools of the trade	Load balancers, message queues, caches	OOP, SOLID, design patterns, UML
Sample question	"Design WhatsApp"	"Design a rate limiter" / "Design a parking lot"

Why LLD matters even outside interviews: the skills are the same ones that keep a real codebase maintainable — naming the right responsibilities, keeping classes small and cohesive, and reaching for a pattern only when it earns its place.

Where LLD sits in the full design pipeline

1. **Requirements gathering** — functional and non-functional requirements (Chapter 9).
2. **HLD** — pick services, protocols, and storage.
3. **LLD** — inside each service, design the object model: classes, interfaces, relationships.
4. **Implementation** — write the actual code, guided by the class diagram from step 3.

The rest of this book lives mostly in step 3 — with a full Part dedicated to putting it into practice on sixteen real interview questions, each solved file-by-file.

2.1 Encapsulation: own the state and its rules

Meaning and motivation

Encapsulation puts related state and behavior behind a controlled boundary. A private field alone is insufficient: an unrestricted setter can still let callers create invalid objects. A bank account should offer deposit and withdraw operations, because those names express what the domain permits. The invariant in this example is that balance never becomes negative and every transfer amount is positive. Validate before assignment so a failed operation leaves the object unchanged.

Access control in Java

public members form the API. private members belong to the enclosing top-level class boundary, including its nested classes. With no modifier, package-private access stays within the package. protected grants package access and subclass access; across packages, a subclass may access an inherited protected instance member only through an appropriately typed subclass reference, not through any arbitrary parent object. Top-level classes are public or package-private. Use the narrowest visibility that supports the collaboration. [J3]

Keywords and state ownership

class defines the type; new creates an instance; this identifies the current instance and distinguishes a field from a same-named parameter. final allows a variable to be assigned once: a final List reference can still point to a mutable list. static belongs to the class, so a static balance would accidentally be shared by all accounts. A constructor establishes the initial invariant; throw rejects illegal input. BigDecimal represents decimal amounts without binary floating-point rounding. Use string constructors for monetary literals.

Defensive copying and immutability

A getter that returns an internal mutable list can leak the boundary. Copy input collections on construction and expose immutable snapshots when callers should only observe. List.copyOf protects the list structure, but does not deeply freeze mutable elements. Here history contains immutable strings, so its snapshot is sufficient. The account itself is intentionally mutable and single-threaded; making fields private does not make compound operations thread-safe.

Complete Java example

Folder: lesson-2-1. Save each listing in its own file.

BankAccount.java

```
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public final class BankAccount {
    private final String id;
    private BigDecimal balance;
    private final List<String> history = new ArrayList<>();

    public BankAccount(String id, BigDecimal opening) {
        this.id = Objects.requireNonNull(id);
        Objects.requireNonNull(opening);
        if (id.isBlank() || opening.signum() < 0) {
            throw new IllegalArgumentException("Invalid account");
        }
        this.balance = opening;
    }

    private static void requirePositive(BigDecimal amount) {
        Objects.requireNonNull(amount);
        if (amount.signum() <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
    }
}
```

5.1 Singleton

Intent and motivating problem

A Singleton controls instance creation so one instance is available within a defined scope. That scope matters: a JVM class loader can have its own copy, and separate processes have separate instances. The pattern does not create one object across a cluster. Use it when identity or a genuinely shared process-level facility is part of the requirement, not merely because an object is convenient to access globally.

Participants and collaboration

AppSettings is an enum with one constant, INSTANCE. The JVM controls enum construction and initialization. Main is the client and reads immutable configuration through methods. There is no public constructor or mutable global map. The example uses immutable state so readers do not need additional coordination. An enum singleton is compact and avoids many serialization and reflective-construction pitfalls of ordinary singleton classes.

When the pattern earns its place

Process-wide immutable metadata is a reasonable teaching use. An initialization-on-demand holder is another Java option: a nested static holder owns the instance and class initialization supplies safe publication. Double-checked locking requires a volatile instance field; omitting it can permit unsafe publication. Framework singleton scope is container-specific and should not be confused with the GoF global-access pattern.

Complete Java example

Folder: lesson-5-1. Save each listing in its own file.

AppSettings.java

```
public enum AppSettings {  
    INSTANCE;  
    private final String applicationName = "LLD Learning App";  
    public String applicationName() { return applicationName; }  
}
```

Main.java

```
public class Main {  
    public static void main(String[] args) {  
        AppSettings first = AppSettings.INSTANCE;  
        AppSettings second = AppSettings.INSTANCE;  
        System.out.println("Same instance: " + (first == second));  
        System.out.println(first.applicationName());  
    }  
}
```

PART 6 — SIXTEEN FAANG LLD CASE STUDIES (60-MINUTE FORMAT)

Chapter 10: Movie Ticket Booking System (BookMyShow-like)

10.1 Problem Statement

Design a movie ticket booking system similar to BookMyShow. The system should allow a user to search for movies playing in a city, view showtimes at various theatres/screens, select seats for a specific show, and book tickets for those seats. The system must prevent two users from booking the same seat for the same show at the same time.

10.2 Functional Requirements

- Support multiple **cities**, each with multiple **theatres**, each theatre having multiple **screens**.
- Each screen has a fixed **seating layout** with seat categories: `SILVER`, `GOLD`, `PLATINUM`, each with a different price.
- A **movie** can have multiple **shows** scheduled across different screens/theatres/times.
- User can **search movies** by city, date, and optionally by movie name/language/genre.
- User can view **seat availability** for a chosen show.
- User can **select and book** one or more seats, generating a booking/ticket with a unique ID.
- Support **seat locking**: once selected, seats are held for ~5 minutes so no one else can book them concurrently; the lock expires if payment isn't completed.
- Support **cancellation**, releasing seats back to availability.
- Compute total **price** based on seat categories selected.

10.3 Non-Functional Requirements

- **Concurrency-safety**: no two threads/users should ever book the same seat for the same show.
- **Extensibility**: adding a seat category or a new city/theatre shouldn't require touching core booking logic (OCP).
- Seat layout must vary per screen, not be hardcoded.
- Favor composition over inheritance for Theatre → Screen → Seat.

10.4 Entities, Relationships & Pattern Rationale

A `City` aggregates `Theatres`; a `Theatre` *composes* its `Screens` (a screen cannot exist without its theatre); a `Screen` composes its physical `Seat` layout. A `Show` associates one `Movie` with one `Screen` at a time slot, and for every show we materialize one `ShowSeat` per physical seat — this is the key modelling decision: the physical `Seat` is reusable across shows, but its *booking status* (`AVAILABLE/LOCKED/BOOKED`) is per-show, so that state lives on `ShowSeat`, not on `Seat` itself.